

SCRIPT D'ORAL — 35 MINUTES

CyberAudit & Solutions

Script complet, diapositive par diapositive, en phrases courtes et faciles à dire — RNCP
5 Développeur Web et Web Mobile

Tommy JOUHANS — Session 2026

16 diapositives • Durée totale visée : 35 min

AVANT DE COMMENCER

Comment utiliser ce script

Ce document reprend, diapositive par diapositive, un script en phrases courtes, pensées pour être dites à voix haute plutôt que lues comme un texte écrit. Chaque phrase peut être mémorisée séparément : pas besoin d'apprendre le texte mot pour mot, l'important est d'enchaîner les idées dans l'ordre.

Les phrases entre crochets, en violet, sont des repères d'action pendant la démonstration en direct (diapositive 9) — elles ne sont pas à dire à voix haute, ce sont des indications pour vous-même.

Conseils de débit

- Le minutage de chaque diapositive suppose un débit de parole posé, avec de courtes pauses entre les phrases — comptez environ 120 à 140 mots par minute.
- Chronométrez au moins deux répétitions complètes à voix haute pour ajuster votre propre rythme : ce script est un guide, pas une camisole.
- Sur la diapositive 9 (démonstration), le temps inclut la navigation dans l'application, pas seulement la parole — prévoyez une marge si le réseau est lent le jour J.
- En cas de trou de mémoire, revenez au mot-clé de la diapositive affichée à l'écran : le déroulé visuel du support vous guide déjà dans l'ordre des idées.

N°	Diapositive	Durée
1	Page de titre	1 min
2	Le problème	2 min 15
3	Notre solution	3 min
4	Contexte et rôles	1 min 30
5	Stack technique	2 min 15
6	Architecture en couches	2 min 30
7	Extrait de code : front-end	2 min 30
8	Extrait de code : back-end	2 min 30
9	Démonstration en direct	4 min 30
10	Modèle de données	2 min 15
11	Parcours client	1 min 30
12	Portail administrateur	1 min 30
13	Déploiement et infrastructure	1 min 30
14	Sécurité	2 min 15
15	Rétrospective	2 min 30
16	Et ensuite ?	1 min 30
Total		35 min



Bonjour à toutes et à tous.

Je m'appelle Tommy Jouhans.

Aujourd'hui, je vais vous présenter mon projet de portfolio : CyberAudit & Solutions.

C'est une plateforme qui permet à des petites et moyennes entreprises de commander un audit de cybersécurité, et de suivre son traitement du début à la fin.

J'ai développé ce projet en binôme, avec James Roussel, dans le cadre de ma formation de Développeur Web et Web Mobile, à Holberton School Dijon, en lien avec l'association Les Entrep' Bourgogne.

Sur ce projet, j'ai été responsable de toute la partie front-end. James s'est occupé de la partie back-end.

Je vais vous présenter le contexte, les choix techniques, la sécurité, et je vous montrerai l'application en fonctionnement.

On commence.

Avant de vous présenter la solution, je veux d'abord poser le problème.

Aujourd'hui, les petites et moyennes entreprises sont des cibles très fréquentes pour les cyberattaques.

Mais elles n'ont presque jamais les moyens de s'en protéger correctement.

Selon l'ANSSI, près de la moitié des PME françaises n'ont aucune équipe informatique dédiée en interne.

Et selon le baromètre annuel du CESIN, la part des entreprises touchées par une cyberattaque augmente chaque année.

Concrètement, ça donne quatre problèmes très concrets sur le terrain.

Premièrement, il n'y a aucune sécurité informatique structurée.

Deuxièmement, quand un audit existe, il est souvent manuel, long, et coûteux à réaliser.

Troisièmement, en cas d'incident, la capacité de réaction est très lente.

Et quatrièmement, il n'y a pas d'outil de reporting simple, compréhensible par un dirigeant qui n'est pas technicien.

C'est exactement ce vide-là que nous avons voulu combler avec CyberAudit & Solutions.

Face à ce problème, nous avons construit une plateforme autour de six fonctionnalités principales.

La première, c'est un formulaire de demande d'audit. Le client choisit une formule : Audit, Security, Protection, ou Premium.

La deuxième, c'est un tableau de bord en temps réel. Le client peut suivre l'avancement de sa demande, à tout moment.

La troisième, c'est un espace de gestion pour l'administrateur. Il peut traiter les demandes, ajouter des notes internes, et changer le statut d'un dossier.

La quatrième, c'est la génération automatique d'un rapport PDF. Il contient un score de sécurité, la liste des vulnérabilités trouvées, et un plan d'action sur trente jours.

La cinquième, ce sont des modules de formation. Des modules courts, pour sensibiliser les équipes du client : reconnaître un email de phishing, activer une authentification à plusieurs facteurs, et ainsi de suite.

Et la sixième, c'est l'authentification. Elle repose sur des jetons JWT, avec une rotation systématique, pour plus de sécurité.

Ces six briques, ensemble, forment un parcours complet : de la demande du client, jusqu'à la remise d'un rapport exploitable.

Ce projet s'est déroulé sur douze semaines, du 27 avril au 17 juillet 2026.

Nous avons travaillé en méthode agile, avec des sprints, une priorisation MoSCoW, et un point de synthèse chaque vendredi.

La répartition des rôles était claire.

J'ai pris en charge tout le périmètre front-end : maquettage, interfaces, logique dynamique, sécurité côté client, et accessibilité.

James Roussel s'est occupé du back-end : l'API, la base de données, l'authentification, et les tâches asynchrones.

Sur les quarante points de complexité prévus au départ, nous en avons livré trente-huit. Soit une vélocité de 95 pourcent.

Les deux points restants concernaient des tests de bout en bout, que nous avons reportés à une itération suivante.

Parlons maintenant de la stack technique.

Côté front-end, j'ai utilisé React 19, l'outil de build Vite, et Tailwind CSS pour le style. Et j'ai ajouté DOMPurify, pour assainir tout contenu affiché, et me protéger des attaques XSS.

Côté back-end, James a utilisé Django, avec Django REST Framework. L'authentification repose sur SimpleJWT. Et un worker Celery gère les tâches asynchrones, comme la génération des rapports PDF avec WeasyPrint.

Pour les données, nous utilisons PostgreSQL, et Redis comme courtier de messages.

Et pour l'infrastructure, le front-end est déployé sur Vercel, le back-end sur Railway, avec un pipeline d'intégration continue GitHub Actions.

Ce pipeline exécute, à chaque poussée de code, cinquante-cinq tests back-end avec pytest, et cinquante-six tests front-end avec Vitest.

Voici le schéma d'architecture de l'application.

Le navigateur du client exécute une application React, en une seule page.

Elle communique en HTTPS avec deux destinations différentes.

D'un côté, le CDN de Vercel, pour récupérer les fichiers statiques.

De l'autre, l'API Django, hébergée sur Railway, pour toutes les opérations authentifiées, sécurisées par un jeton JWT.

L'API dialogue avec la base PostgreSQL grâce à l'ORM de Django.

Et pour les traitements longs, comme la génération d'un rapport PDF, ou l'envoi d'un email, elle délègue le travail à un worker Celery, de façon asynchrone, via une file de messages Redis.

Ce découplage entre le front et le back a plusieurs avantages : deux déploiements indépendants, un contrôle fin des origines autorisées avec CORS, et une mise en cache efficace des ressources statiques.

Je vous montre maintenant deux extraits de code, représentatifs de mon travail côté front-end.

Le premier, à gauche, c'est le composant `ProtectedRoute`. C'est lui qui protège l'accès aux pages selon le rôle de l'utilisateur.

Si la personne n'est pas connectée, elle est redirigée vers la page de connexion. Si elle n'a pas le bon rôle — par exemple, un client qui tente d'accéder à une page réservée à l'administrateur — elle est redirigée vers l'accueil.

Le deuxième extrait, à droite, c'est la fonction `sanitize`. Elle s'appuie sur la bibliothèque `DOMPurify`.

Chaque donnée saisie par un utilisateur, et réaffichée ailleurs dans l'application, passe par cette fonction avant d'être insérée dans la page.

Et la configuration est volontairement stricte : aucune balise HTML, aucun attribut n'est conservé.

C'est ce mécanisme qui a neutralisé, sans aucun problème, la tentative d'injection XSS que nous avons testée dans notre jeu d'essai.



Côté back-end, voici deux extraits que je trouve particulièrement représentatifs.

À gauche, la méthode qui génère la référence unique d'une demande d'audit, au format DOSSIER, année, numéro.

Elle utilise `select_for_update`, pour verrouiller la ligne pendant toute la transaction. Ça garantit que deux demandes, soumises à la même seconde, ne pourront jamais recevoir la même référence.

À droite, la classe de permission `IsAdminOrOwner`. C'est elle qui gère le contrôle d'accès par rôle, côté serveur.

Un administrateur a accès à tout. Un client, uniquement à ses propres demandes.

Et surtout, si l'accès est refusé, la vue renvoie un code 404, et non 403.

L'objectif, c'est de ne jamais révéler à un attaquant qu'une ressource existe, même s'il n'a pas le droit d'y accéder.

Je vous propose maintenant de voir l'application en fonctionnement.

[Ouvrir le lien de démonstration : portfolio-kappa-dun-65.vercel.app]

Je me connecte ici avec un compte client. Je vais soumettre une nouvelle demande d'audit.

Je choisis une formule — par exemple, la formule Audit. Et je remplis les notes de périmètre : par exemple, « audit du serveur de fichiers et de douze postes Windows ». Je valide.

Vous voyez que la demande apparaît immédiatement dans mon tableau de bord, avec le statut « en attente ».

Et elle reçoit une référence unique, au format DOSSIER, année, numéro — celle-là même que nous venons de voir générée dans le code, à l'instant.

Je bascule maintenant côté administrateur. [Se déconnecter, se reconnecter avec un compte admin]

La demande apparaît dans la file de traitement. Je peux l'assigner à un opérateur, changer son statut — par exemple « en attente » vers « en cours » — et ajouter une note interne.

Je déclenche la génération du rapport PDF. Cette génération est traitée de façon asynchrone, par le worker Celery. Ça veut dire que je peux continuer à travailler pendant que le PDF se génère en arrière-plan.

Une fois le rapport prêt, le client reçoit une notification par email, et peut le télécharger directement depuis son tableau de bord. [Revenir sur le compte client, télécharger le rapport]

Voilà : vous venez de voir le parcours complet, de la demande initiale du client, jusqu'à la remise du rapport final.

Repère — Prévoir une capture d'écran ou une courte vidéo de secours, à ouvrir immédiatement en cas de souci réseau le jour J.

Voici le modèle de données de l'application.

La base PostgreSQL s'organise autour de sept entités principales.

L'utilisateur, qui peut être client ou administrateur. Le catalogue des formules d'audit. La demande d'audit elle-même, avec sa référence unique, générée de façon atomique, comme je viens de vous le montrer.

Le rapport d'audit, avec son score de sécurité, et la liste des vulnérabilités, au format JSON. Les modules de formation, et la progression de chaque utilisateur sur ces modules. Et enfin, les notifications.

Un choix que je veux souligner : toutes les clés primaires sont des UUID, et non des entiers auto-incrémentés.

C'est un choix de sécurité délibéré. Ça empêche un attaquant de deviner l'existence d'autres ressources, simplement en incrémentant un identifiant dans l'URL.

Le parcours côté client a été pensé pour rester simple, pour un utilisateur non technicien.

C'est le persona principal du projet : Marie, dirigeante de PME, peu à l'aise avec l'informatique.

Depuis la page d'accueil, elle s'inscrit, ou se connecte. Elle arrive sur un tableau de bord, qui liste ses demandes, avec un statut coloré.

Elle peut soumettre une nouvelle demande, consulter le détail d'une demande existante avec son historique complet, et télécharger son rapport dès qu'il est disponible.

Ce parcours a été validé avant même le début du développement des interfaces.

Côté administrateur, l'application propose un tableau de bord global.

On peut filtrer l'ensemble des demandes par statut, et ouvrir le détail d'une demande pour consulter les informations du client.

Changer le statut, assigner la demande, ajouter des notes internes.

Déclencher la génération du rapport PDF en un clic, envoyer une notification au client, et archiver une demande une fois qu'elle est traitée.

Un point que je veux souligner : chaque action est consignée dans un historique immuable. Ça garantit la traçabilité, exigée par le RGPD.

Le déploiement suit une démarche d'intégration et de livraison continues.

À chaque poussée de code, GitHub Actions exécute l'analyse statique du code, la suite de tests, et déclenche le déploiement.

Le front-end est reconstruit et publié sur le CDN mondial de Vercel.

Le back-end est déployé sur Railway, à partir d'un conteneur Docker. Les migrations de base de données sont appliquées automatiquement, et un processus séparé exécute le worker Celery.

Les identifiants sensibles ne figurent jamais dans le dépôt de code.

Et une sauvegarde manuelle de la base de production est déclenchée avant chaque mise en production majeure, avec une rétention de quatorze jours.

La sécurité a été traitée comme une exigence transversale, présente dès la conception.

Vous venez de voir deux extraits de code qui l'illustrent très concrètement.

Côté front-end : assainissement systématique contre les attaques XSS, en-têtes de sécurité HTTP sur chaque réponse, et jetons stockés en sessionStorage.

Côté back-end : l'ORM Django élimine par construction le risque d'injection SQL. Le contrôle d'accès par rôle renvoie un 404, plutôt qu'un 403. Et les points d'entrée sensibles sont protégés par une limitation de débit.

Pour vérifier concrètement l'efficacité de toutes ces mesures, nous avons mené un audit manuel de sécurité.

Six scénarios d'attaque testés : injection XSS, injection SQL, force brute, falsification de jeton, élévation de privilège, et énumération de ressources.

Résultat : aucune faille exploitable n'a été identifiée.

Pour conclure sur la méthode, je voudrais partager notre rétrospective d'équipe. Nous faisons ce point chaque vendredi, avec James.

Ce que nous avons choisi de continuer : les revues de code croisées sur chaque pull request, la veille sécurité hebdomadaire, et le réflexe d'intégrer la sécurité dès la conception, plutôt qu'à la fin.

Ce que nous avons choisi d'arrêter : discuter les contrats d'interface entre le front et le back uniquement à l'oral, sans les tracer par écrit — ça nous a coûté des allers-retours évitables. Et sous-estimer le temps que prennent réellement les tests de bout en bout — c'est d'ailleurs pour ça que nous avons reporté les deux points de tests Cypress.

Et ce que nous ferions différemment, sur un prochain projet : documenter chaque contrat d'interface par écrit avant de commencer à coder, et provisionner du temps pour les tests de bout en bout dès la planification du sprint.

Au final, sur ces douze semaines : 95 pourcent du périmètre livré, et zéro bogue ouvert à l'issue de chaque sprint.

Pour terminer, voici quelques pistes d'évolution pour une future version du produit.

Un tableau de bord réellement temps réel, avec WebSocket. Un calcul de score de sécurité assisté par intelligence artificielle. Une déclinaison en application mobile native.

La gamification des modules de sensibilisation. Une authentification déléguée, via Google ou LinkedIn. Et un journal d'audit applicatif complet, pour renforcer encore la traçabilité RGPD.

Je vous remercie de votre attention.

Et je reste à votre disposition pour toutes vos questions.